



香港中文大學 (深圳)  
The Chinese University of Hong Kong

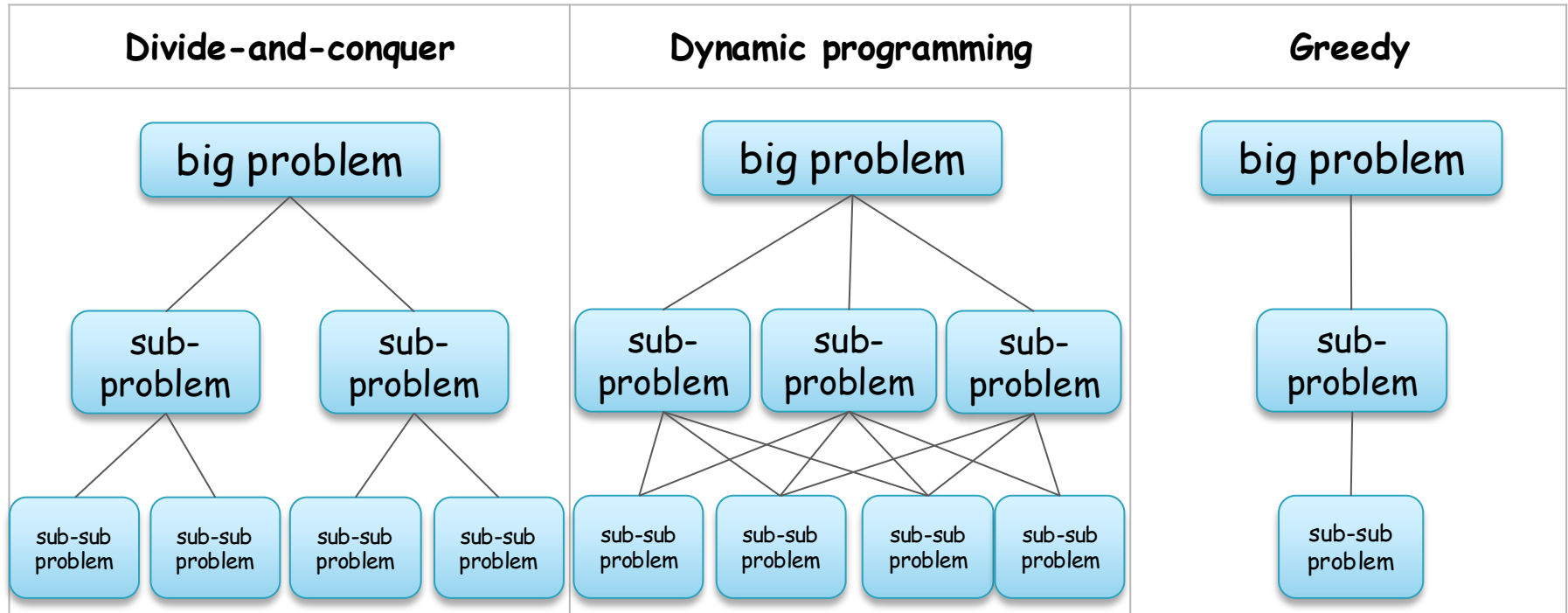
# CSC 5003 Algorithm Art and Programming Practice Lecture 10: Greedy and Divide-and-Conquer

Jingbang Chen  
School of Data Science (SDS)  
The Chinese University of Hong Kong, Shenzhen

---



# Differences among Greedy, DP, and DC





# Candy

---

- ▶ There are  $n$  children standing in a line.
  - ▶ Each child has a rating given by an array ratings.  
You must give candies to the children such that:
    - Each child gets at least 1 candy.
    - If a child has a higher rating than an adjacent child, they must get more candies
- Return the minimum total number of candies.



# Solution

---

- ▶ The constraints are local (adjacent only), but the objective is global minimum.
- ▶ Trick: split constraints into two directions
  - Left constraint (compare with left neighbor)
  - Right constraint (compare with right neighbor)



# Solution

---

## Step 1 — Left to Right

Let `left[i]` = minimum candies needed to satisfy:

- each child  $\geq 1$
- and if `ratings[i] > ratings[i-1]`, then `candy[i] > candy[i-1]`

Then:

$$left[i] = \begin{cases} left[i-1] + 1 & \text{if } ratings[i] > ratings[i-1] \\ 1 & \text{otherwise} \end{cases}$$

---

## Step 2 — Right to Left

Let `right[i]` = minimum candies needed to satisfy:

- each child  $\geq 1$
- and if `ratings[i] > ratings[i+1]`, then `candy[i] > candy[i+1]`

$$right[i] = \begin{cases} right[i+1] + 1 & \text{if } ratings[i] > ratings[i+1] \\ 1 & \text{otherwise} \end{cases}$$



# Solution

---

## Step 3 — Combine

To satisfy **both constraints**, we must have:

$$candy[i] = \max(left[i], right[i])$$

- ▶ This is a **canonical greedy pattern**:
  - When constraints come from **both directions**:
    1. Solve each direction independently (get minimal requirements)
    2. Take the **maximum per position**



# Strange Test

---

- ▶ You are given two integers  $a$  and  $b$ , with  $a < b$ .
- ▶ In one move, you may do **one** of the following:
  1. Increase  $a$  by 1
  2. Increase  $b$  by 1
  3. Replace  $a$  by  $a | b$  (bitwise OR)
- ▶ Your goal is to make  $a = b$ .
- ▶ Return the **minimum number of moves**.



# Solution

---

## First Observations:

- ▶ A trivial solution is always: Increase  $a$  until it becomes  $b$
- ▶ This takes  $b-a$  moves.
- ▶ So the answer is at most  $b-a$ .
- ▶ The interesting part is whether using the OR operation can do better.





# Solution

---

- ▶ It is optimal to apply the third operation at most once, because it does not decrease  $a$  and always makes  $b \leq a$
- ▶ This means that after we use it, we can only apply the second operation.
- ▶ If we don't apply the third operation, the answer is  $b-a$ .



# Solution

- ▶ Suppose we do apply it. Before that, we used the first and second operations some number of times, let the resulting values of  $a$  and  $b$  be  $a'$  and  $b'$  respectively ( $a \leq a', b \leq b'$ ).
- ▶ The answer in this case will be  $(a' - a) + (b' - b) + ((a' \mid b') - b') + 1 = a' + (a' \mid b') + (1 - a - b)$ . This is equivalent to minimizing  $a' + (a' \mid b')$ , since  $(1 - a - b)$  is constant.
- ▶ To do that, we can iterate  $a'$  from  $a$  to  $b$ . For a fixed  $a'$ , we have to minimize  $a' \mid b'$ , the optimal  $b'$  can be constructed like this:
  - Set  $b'$  to zero and iterate over bits from highest to lowest. There are 4 cases:
    - If current bit of  $a'$  is 0 and  $b$  is 1, set the current bit of  $b'$  to 1.
    - If current bit of  $a'$  is 0 and  $b$  is 0, set the current bit of  $b'$  to 0.
    - If current bit of  $a'$  is 1 and  $b$  is 1, set the current bit of  $b'$  to 1.
    - If current bit of  $a'$  is 1 and  $b$  is 0, set the current bit of  $b'$  to 1 and stop.



# Score Maximization

---

You are given:

- an array  $a$  of length  $n$ ,
- an integer  $k$ ,
- a binary array  $b$  of length  $n$ .

You may perform the following operation at most  $k$  times:

- choose an index  $i$  with  $b_i = 1$ ,
- increase  $a_i$  by 1.

For each index  $i$ , let  $c_i$  be the array obtained by deleting  $a_i$  from  $a$ .

Your score is

$$\max_{1 \leq i \leq n} \left( a_i + \text{median}(c_i) \right).$$

Here the median of an array of length  $m$  is the  $\lfloor \frac{m+1}{2} \rfloor$ -th smallest element.

You want the **maximum possible score** after at most  $k$  operations.



# Solution

---

Since the score depends only on the multiset of values, we may sort  $a$  first:

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

Now ask: for a fixed index  $i$ , what can  $\text{median}(c_i)$  be?

The editorial observation is that, after deleting one element, the median of the remaining  $n - 1$  elements is always one of only two positions of the sorted array: either

$$a_{\lfloor n/2 \rfloor} \quad \text{or} \quad a_{\lfloor (n+2)/2 \rfloor}.$$

More precisely:

- if  $i \leq \lfloor n/2 \rfloor$ , then

$$\text{median}(c_i) = a_{\lfloor (n+2)/2 \rfloor},$$

- otherwise,

$$\text{median}(c_i) = a_{\lfloor n/2 \rfloor}.$$



# Solution

---

Key consequence:

- ▶ For a fixed sorted array, the best score is always achieved by taking the maximum element  $a_n$  as the chosen  $a_i$ .
- ▶ So the score of the final array is:
  - $a_n + \text{median}(c_n)$   
that is, maximum element + median of the other  $n-1$  elements.



# Solution

---

- ▶ In an optimal strategy, either
  1. we spend all useful operations on the element that becomes the maximum, or
  2. we keep the maximum fixed and spend operations to improve the median of the remaining elements.



# Solution

---

Suppose we choose some index  $i$  with  $b_i = 1$ , and after all operations this element is the one contributing as the max.

If we spend all  $k$  operations on it, its contribution becomes:

$$a_i + k.$$

The other term is  $\text{median}(c_i)$ , which depends only on deleting that element from the sorted array.

If the sorted position of this chosen element is  $i$ , then:

- if  $i < n/2$ , the deleted element lies in the left half, so

$$\text{median}(c_i) = a_{n/2},$$

- otherwise,

$$\text{median}(c_i) = a_{(n-2)/2}.$$

So this case gives candidate values

$$a_i + k + \text{median}(c_i) \quad \text{for all } i \text{ with } b_i = 1.$$



# Solution

---

Now we keep  $a_n$  as the maximum and use operations only on the other  $n - 1$  elements, trying to maximize

$$\text{median}(c_n).$$

So the problem becomes:

Among the first  $n - 1$  elements, after at most  $k$  increments on positions with  $b_i = 1$ , what is the largest median we can achieve?

► Binary Search!





# How to check whether a target median $x$ is achievable?

---

Let us test whether we can make

$$\text{median}(c_n) \geq x.$$

The array  $c_n$  has length  $n - 1$ .

Its median is the  $\lfloor \frac{n}{2} \rfloor$ -th smallest element, which means we need at least

$$\left\lfloor \frac{n+1}{2} \right\rfloor$$

elements in  $c_n$  to be at least  $x$ . This is the threshold used in the official solution.

So we count:

- elements already  $\geq x$ ,
- plus elements  $< x$  with  $b_i = 1$  that we can raise to  $x$ .

If  $a_i < x$  and  $b_i = 1$ , the cost to make it reach  $x$  is

$$x - a_i.$$



# How to check whether a target median $x$ is achievable?

---

- ▶ To maximize how many elements we can raise under budget  $k$ , we should pick the **largest**  $a_i$  below  $x$  first, because they need the fewest increments.
- ▶ Thus the feasibility check is:
  - sort the candidates below  $x$  in descending order of  $a_i$ ,
  - greedily raise as many as possible,
  - see whether the total number of elements  $\geq x$  reaches  $\lfloor (n+1)/2 \rfloor$ .
- ▶ Then binary search the largest feasible  $x$ .



# Solution

---

For each test case:

1. Sort pairs  $(a_i, b_i)$  by  $a_i$ .
2. Compute **Case 1**:
  - for every  $i$  with  $b_i = 1$ ,
  - candidate  $= a_i + k + \text{median}(c_i)$ .
3. Compute **Case 2**:
  - binary search the largest median  $x$  achievable among the first  $n - 1$  elements,
  - candidate  $= a_n + x$ .
4. Take the maximum of the two cases.



# Proof

**Claim 1 :** The score of the final array (after sorting in increasing order) is  $a_n + \text{med}(c_n)$

▼ Proof 1

There are **2** cases we need to consider.

1)  $\text{med}(c_i) = a_{\lfloor \frac{n}{2} \rfloor}$ , in which case optimum value of  $i$  is  $n$  as we want to maximise  $a_i$ . Thus score in this case is  $a_n + a_{\lfloor \frac{n}{2} \rfloor}$ .

2)  $\text{med}(c_i) = a_{\lfloor \frac{n+2}{2} \rfloor}$ , in which case optimum value of  $i$  is  $\lfloor \frac{n}{2} \rfloor$  as this is the largest value of  $i$  which will change median. This obtains a score of  $a_{\lfloor \frac{n}{2} \rfloor} + a_{\lfloor \frac{n+2}{2} \rfloor}$

The score in Case **1** is clearly larger than Case **2**, hence it is optimal.

**Claim 2 :** Either we will use all  $k$  operations on the element which eventually becomes max element in our array, or we will use all operations trying to improve  $\text{med}(c_n)$  and keep max element constant.

▼ Proof 2

Suppose we did  $x$  ( $0 < x < k$ ) operations on the element which eventually became maximum.

Then, we could have done the remaining  $(k - x)$  operations on this element too, as this is already the max element and doing operations on it guaranteedly improves our score each time by **1**. Doing operations on any other element can only improve our score by atmost **1**



# Take a break...

---



# Count of Smaller Numbers After Self

---

- ▶ You are given an integer array `nums`.
- ▶ For each index  $i$ , you need to count how many numbers to the right of  $i$  are smaller than `nums[i]`.



# Solution

---

- ▶ Do we still remember how to code mergesort?
- ▶ How to code mergesort to compute inversions?
- ▶ Now we simply want to compute the contribution for each index.



# Solution

---

- ▶ During merge sort, when we merge two sorted halves:
  - left half = elements that originally appear earlier
  - right half = elements that originally appear later
- ▶ So if an element in the left half is greater than some elements in the right half, then those right-half elements are exactly “smaller numbers after self” for that element.
- ▶ Thus, during merging, we can count how many elements from the right half move before each left-half element.





# Solution

---

- ▶ We store not only the values, but also their original indices. Let each element be a pair: (value, index)
- ▶ We maintain an answer array `ans[index]`.
- ▶ Suppose we merge two sorted halves:
  - `left = arr[l..mid]`, `right = arr[mid+1..r]`
- ▶ We use two pointers.
- ▶ When `left[i].value <= right[j].value`, then `left[i]` is placed first.
- ▶ At this moment, some number of elements from the right half may already have been placed before it. Those are exactly the right-half elements smaller than `left[i]`. So we add that number to: `ans[left[i].index]`



# Count of Range Sum

---

- ▶ You are given an integer array `nums` and two integers `lower` and `upper`.
- ▶ Return the number of range sums lying in `[lower, upper]`, inclusive.



# Solution

---

## ► Transform to Prefix Sum:

So the problem becomes:

Count pairs  $(i, j)$  with  $i < j$  such that

$$lower \leq pre[j] - pre[i] \leq upper$$

Equivalently, for each  $j$ , count how many earlier  $i$  satisfy:

$$pre[j] - upper \leq pre[i] \leq pre[j] - lower$$



# Solution

---

- ▶ Suppose we recursively sort prefix sums.
- ▶ When merging two sorted halves:
  - left half = prefix sums with smaller original indices
  - right half = prefix sums with larger original indices
- ▶ So every valid pair across the two halves automatically satisfies index order.
- ▶ Now for each value in the left half, we want to count how many values in the right half lie in:
  - $[\text{leftValue} + \text{lower}, \text{leftValue} + \text{upper}]$
- ▶ Since the right half is sorted, this can be done with two pointers.



# Solution

---

For each  $pre[i]$  in the left half, we want all  $pre[j]$  in the right half such that:

$$lower \leq pre[j] - pre[i] \leq upper$$

That is:

$$pre[i] + lower \leq pre[j] \leq pre[i] + upper$$

Because the right half is sorted, we maintain two pointers:

- $j1$  = first position with

$$pre[j1] - pre[i] \geq lower$$

- $j2$  = first position with

$$pre[j2] - pre[i] > upper$$

So the number of valid  $j$  is:

$$j2 - j1$$